

PCA Report

Student name: Van Thinh TO
Student ID: 22518830

Contents

1	Set up	2
2	Session 1 — PCA	2
3	Session 2 — Hierarchical clustering and linkage	6
4	Session 3 — Choosing k and bootstrap stability	8
5	Session 4 — Clustering, PCA, and heatmap	11
6	Going further — Personal reflection	15

1 Set up

```
knitr::opts_chunk$set(
  echo = TRUE, message = FALSE, warning = FALSE,
  fig.align = "center", fig.pos = "H",
  out.width = "72%", dpi = 200, dev = "pdf", comment = "##"
)

options(digits = 4)

library(FactoMineR)
library(cluster)
library(fpc)
library(pheatmap)

shape_names <- c("rEV21", "rEV32", "rEV31")
five_names <- c(shape_names, "Num_Atoms", "Rg")
cluster_colors <- c("#2C7FB8", "#D95F02", "#33A02C", "#8C1D40")
origin_colors <- c(Prot = "#2C7FB8", RNA = "#8C1D40")
```

2 Session 1 — PCA

PCA combines measurements into new axes. PC1 captures the most variation; PC2 captures the most remaining variation in a perpendicular direction. **Variance** measures spread between pockets. **Scores** are their coordinates on these axes. PCA centers the measurements automatically; scaling also divides them by their standard deviations (SDs).

2.1 Import and check the data

We inspect column types, missing values, and Origin counts before analysis.

```
# Read the file and show a short preview of each column.
d1 <- read.csv("data_NS.csv", stringsAsFactors = TRUE)
str(d1, vec.len = 2, strict.width = "cut", width = 75)
```

```
## 'data.frame':    600 obs. of  10 variables:
## $ PDB_ID      : Factor w/ 600 levels "ID_001","ID_002",...: 498 464 504 303 ..
## $ Origin      : Factor w/ 2 levels "Prot","RNA": 1 1 1 1 1 ...
## $ Pocket      : Factor w/ 564 levels "ODU_4000-B","OEC_101-A",...: 7 347 192..
## $ Eigval.1    : num  28.7 31.9 ...
## $ Eigval.2    : num  20.3 15.6 ...
## $ Eigval.3    : num  10.3 11 ...
## $ rEV21       : num  0.709 0.49 ...
## $ rEV32       : num  0.508 0.705 ...
## $ rEV31       : num  0.36 0.345 ...
## $ Num_Atoms   : int   135 117 160 180 132 ...
```

```
# Count missing entries and pockets of each Origin.
colSums(is.na(d1))
```

```
##      PDB_ID      Origin      Pocket      Eigval.1      Eigval.2      Eigval.3      rEV21      rEV32
##          0          0          0          0          0          0          0          0
##      rEV31 Num_Atoms
##          0          0
```

```
table(d1$Origin)
```

```
##  
## PROT  RNA  
##  300  300
```

Reading the data: PDB_ID and Pocket are identifiers; Origin is RNA or protein. Eigval.1, .2, and .3 describe atomic spread along the largest, intermediate, and smallest pocket axes. The ratios are $rEV21 = \lambda_2/\lambda_1$, $rEV32 = \lambda_3/\lambda_2$, and $rEV31 = \lambda_3/\lambda_1$; they describe shape proportions. Num_Atoms counts atoms and describes size. The missing-value output gives one count per column; the Origin output gives group counts.

Answer: There are 600 pockets, 10 columns, 300 RNA and 300 protein pockets, and no missing values. Identifiers are excluded from PCA.

2.2 Shape-only PCA, without scaling

We use the three ratios to study shape alone. `scale.unit = FALSE` keeps their original scales. `quali.sup = 4` makes the fourth column, Origin, supplementary: it labels groups without determining the axes.

```
# Calculate PCA from the three shape ratios only.  
shape_input <- d1[c(shape_names, "Origin")]  
pca1 <- PCA(shape_input, scale.unit = FALSE, quali.sup = 4, graph = FALSE)  
pca1$eig
```

```
##          eigenvalue percentage of variance cumulative percentage of variance  
## comp 1  0.0486643          59.7435          59.74  
## comp 2  0.0321893          39.5177          99.26  
## comp 3  0.0006018           0.7389         100.00
```

Table columns: eigenvalue is the variance captured by that PC; percentage of variance is its share of the total; cumulative percentage adds the shares from PC1 through that row. These PCA eigenvalues describe variation across pockets, unlike the geometric eigenvalues in the CSV.

```
# Plot pockets, colored by Origin; nearby points have similar shape scores.  
plot(pca1, choix = "ind", habillage = 4, label = "none",  
     title = "Shape PCA: Origin")
```

Shape PCA: Origin

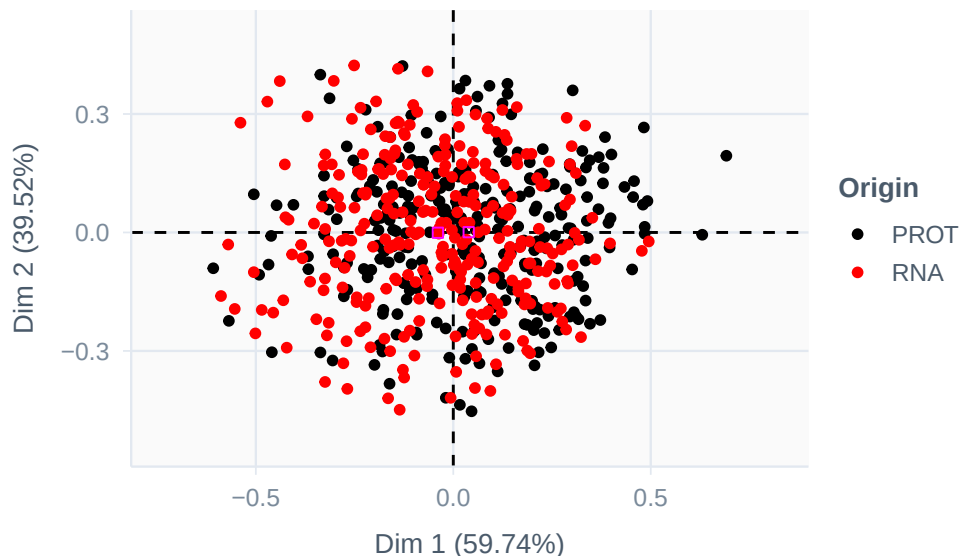


Figure 1: Shape-only PCA with supplementary Origin.

We check redundancy numerically rather than relying on the PCA alone.

```
# Compare ratios and test whether the third is the product of the first two.
```

```
shape_cor <- cor(d1[shape_names])
```

```
shape_cor
```

```
##           rEV21    rEV32  rEV31
## rEV21    1.00000 -0.06497 0.6126
## rEV32   -0.06497  1.00000 0.7158
## rEV31    0.61255  0.71583 1.0000
```

```
redundancy_error <- max(abs(d1$rEV31 - d1$rEV21 * d1$rEV32))
```

```
redundancy_error
```

```
## [1] 0.0001051
```

Correlation table: Rows and columns identify measurements. Their intersection ranges from -1 to +1: positive means increasing together, negative means moving oppositely, and near zero means little linear relation. The diagonal is 1; the two halves repeat the same comparisons.

Answer: PC3 captures only 0.74%; the first two PCs retain 99.26%. The ratios repeat information because $rEV31 = rEV21 \times rEV32$, with maximum discrepancy $1.05e-04$, consistent with rounding. PC3 is not exactly zero because this relation is multiplicative, while PCA captures linear variation. RNA and protein points overlap widely; shape does not clearly separate Origins.

2.3 Add size and compare scaling

We compute $R_g = \sqrt{Eigval1 + Eigval2 + Eigval3}$, which measures atomic spread around the pocket center. We then use the same five measurements in two PCAs, changing only scaling to identify its effect.

```
# Add Rg and run the five-variable PCA before and after scaling.
```

```
d1$Rg <- sqrt(d1$Eigval.1 + d1$Eigval.2 + d1$Eigval.3)
```

```
five_input <- d1[c(five_names, "Origin")]
```

```
pca_raw <- PCA(five_input, scale.unit = FALSE, quali.sup = 6, graph = FALSE)
pca2 <- PCA(five_input, scale.unit = TRUE, quali.sup = 6, graph = FALSE)
```

```
# Check how much raw variance each measurement contributes.
```

```
raw_variance <- numeric(5)
names(raw_variance) <- five_names
for (i in 1:5) {
  raw_variance[i] <- var(d1[[five_names[i]]])
}
data.frame(Variable = five_names, Variance = raw_variance,
           Percent = 100 * raw_variance / sum(raw_variance))
```

```
##           Variable  Variance  Percent
## rEV21          rEV21 2.842e-02 4.057e-04
## rEV32          rEV32 3.402e-02 4.857e-04
## rEV31          rEV31 1.915e-02 2.733e-04
## Num_Atoms Num_Atoms 7.004e+03 9.998e+01
## Rg              Rg 1.472e+00 2.101e-02
```

```
# Compare the individual PCs' explained-variance percentages.
```

```
data.frame(PC = 1:5, Unscaled_percent = pca_raw$eig[, 2],
           Scaled_percent = pca2$eig[, 2])
```

```
##      PC Unscaled_percent Scaled_percent
## comp 1  1          9.999e+01         47.148
## comp 2  2          6.602e-03         28.717
## comp 3  3          5.839e-04         20.766
## comp 4  4          4.315e-04          2.869
## comp 5  5          8.571e-06          0.500
```

Table columns: Variable names the measurement; Variance gives its original spread; Percent is its share of total raw variance. In the comparison, PC identifies the component, and Unscaled_percent and Scaled_percent give its individual share before and after scaling.

```
# Check shape-size relations and interpret the scaled PCA axes.
```

```
cor(d1[five_names])
```

```
##           rEV21    rEV32  rEV31 Num_Atoms      Rg
## rEV21      1.00000 -0.06497 0.6126   0.1822 -0.01081
## rEV32     -0.06497  1.00000 0.7158   0.2823  0.19067
## rEV31      0.61255  0.71583 1.0000   0.3325  0.13755
## Num_Atoms  0.18217  0.28229 0.3325   1.0000  0.82973
## Rg         -0.01081  0.19067 0.1376   0.8297  1.00000
```

```
pca2$var$cor[, 1:2]
```

```
##      Dim.1  Dim.2
## rEV21  0.4352 0.5747
## rEV32  0.6645 0.2212
## rEV31  0.8091 0.5694
## Num_Atoms 0.7980 -0.5019
## Rg      0.6596 -0.6932
```

Dim.1 and Dim.2 are each measurement's correlations with PC1 and PC2, not percentages or PC weights. They explain what the axes represent.

Answer: Without scaling, atom-count variance is 7004.2, versus about 0.02–0.03 for the ratios. It supplies 99.978% of raw variance; PC1 captures 99.992% and mostly describes atom count. This is a numerical-scale artifact.

Scaling gives each variable variance 1. PC1 and PC2 then capture 47.1% and 28.7%. PC1 combines shape and size; PC2 contrasts them. Atom count and Rg are strongly correlated (0.83), while shape-size correlations are weaker.

2.4 Session 1 summary

Same biological information? No: shape describes proportions; size describes atom count and spread. Similar shapes can have different sizes. **Role of scaling?** It stops large numerical scales from dominating, allowing both shape and size to appear. It does not remove redundancy.

3 Session 2 — Hierarchical clustering and linkage

3.1 Reload and compare four dendrograms

Hierarchical clustering repeatedly joins similar groups into a tree. We reload the data and calculate Euclidean distances using only the three unscaled ratios, keeping size out of this session.

```
# Reload shape measurements and calculate pairwise distances.
d2 <- read.csv("data_NS.csv", stringsAsFactors = TRUE)
x2 <- d2[shape_names]
dist2 <- dist(x2)

# Change only the linkage rule, keeping the distances identical.
tree_single <- hclust(dist2, method = "single")
tree_complete <- hclust(dist2, method = "complete")
tree_average <- hclust(dist2, method = "average")
tree_ward <- hclust(dist2, method = "ward.D2")
```

Rules: Single uses the closest pair between groups; complete uses the farthest pair; average uses their average distance; Ward minimizes the increase in within-group squared dispersion.

```
# Display four trees; hide pocket names to keep them readable.
par(mfrow = c(2, 2), mar = c(3, 4, 3, 1))
plot(tree_single, labels = FALSE, main = "Single", xlab = "", sub = "")
plot(tree_complete, labels = FALSE, main = "Complete", xlab = "", sub = "")
plot(tree_average, labels = FALSE, main = "Average", xlab = "", sub = "")
plot(tree_ward, labels = FALSE, main = "Ward", xlab = "", sub = "")
```

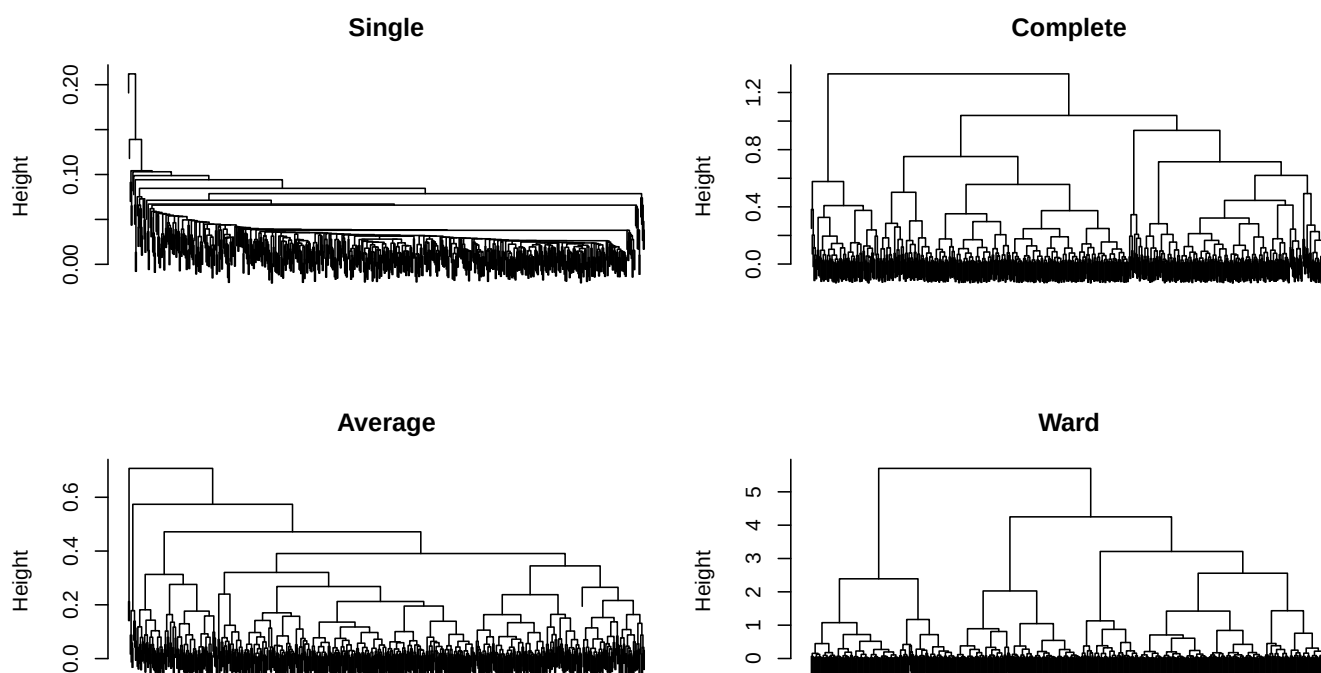


Figure 2: Four linkage rules on identical shape distances.

```
par(mfrow = c(1, 1))
```

Reading: Leaves are pockets; branches record joins. Height is the criterion's joining value, not a pocket measurement. Heights are not directly comparable across criteria. Single shows strong chaining; average has a dominant branch; complete and Ward are more branched. Ward has clearer main branches and large final joins. No partition is chosen from this view alone.

3.2 Cut at $k = 4$: cluster sizes and ARI

`cutree()` assigns groups; `table()` counts their members. Four groups are provisional, as required by the assignment.

```
# Cut each tree into four groups and compare their sizes.
cluster_single <- cutree(tree_single, k = 4)
cluster_complete <- cutree(tree_complete, k = 4)
cluster_average <- cutree(tree_average, k = 4)
cluster_ward <- cutree(tree_ward, k = 4)
size_table <- cbind(single = table(cluster_single),
                    complete = table(cluster_complete),
                    average = table(cluster_average),
                    ward.D2 = table(cluster_ward))

size_table
```

```
##   single complete average ward.D2
## 1    595      289     496     145
## 2      1       80      94     219
## 3      1     214       2     169
## 4      3      17       8      67
```

Table: Rows are group numbers, columns are methods, entries are pocket counts. Each column totals 600. Group numbers need not match across methods. Single gives 595/1/1/3; average 496/94/2/8; complete 289/80/214/17; Ward 145/219/169/67. Ward avoids tiny outlier groups.

What is ARI? The adjusted Rand index asks: do two methods group the same pockets together? It compares pairs of pockets. A pair agrees if both methods put it together, or both put it apart. It disagrees if only one method puts it together. The adjustment accounts for agreement expected by chance, given the group sizes.

- **1:** identical grouping, even if the cluster numbers are renamed.
- **Near 0:** agreement is about what chance would produce, not zero overlap.
- **Negative:** less agreement than expected by chance.

Higher ARI means more similar memberships. It does not show that groups are biologically correct or well separated, and it is not a percentage of matching pockets. We use it here to check how much changing linkage changes the partition. `comparison$corrected.rand` extracts ARI; the loops compare every pair of methods.

```
# Compare every pair of partitions; extract only the ARI.
memberships <- list(cluster_single, cluster_complete,
                    cluster_average, cluster_ward)
methods <- c("single", "complete", "average", "ward.D2")
ari_matrix <- matrix(0, 4, 4)
rownames(ari_matrix) <- methods
colnames(ari_matrix) <- methods
for (i in 1:4) {
  for (j in 1:4) {
    comparison <- fpc::cluster.stats(dist2, memberships[[i]],
                                    memberships[[j]], compareonly = TRUE)
    ari_matrix[i, j] <- comparison$corrected.rand
  }
}
ari_matrix
```

```
##           single complete average  ward.D2
## single    1.000000 0.005995 0.07788 0.001263
## complete  0.005995 1.000000 0.05789 0.316708
## average   0.077880 0.057887 1.00000 0.146254
## ward.D2   0.001263 0.316708 0.14625 1.000000
```

Table: Each entry compares its row and column methods. The diagonal is 1; the two halves repeat comparisons. Complete versus Ward has ARI 0.317: their grouping agrees only partly. Single versus Ward is near zero (0.001), so their agreement is near the chance baseline. This confirms that linkage changes membership.

3.3 Session 2 summary

Which rule and why? Keep Ward: it avoids chaining and produces more reasonably sized, compact groups. This does not validate $k = 4$; we test separation and stability next.

4 Session 3 — Choosing k and bootstrap stability

4.1 Rebuild Ward and compare silhouette for $k = 2$ to 8

We rebuild the same unscaled shape tree. Changing only k lets us compare partitions fairly. A **silhouette** measures how much better a pocket fits its own group than the nearest alternative group: near +1 is well separated, near 0 is a boundary, and negative suggests a better alternative assignment.

```
# Reload and rebuild the same shape clustering.
d3 <- read.csv("data_NS.csv", stringsAsFactors = TRUE)
x3 <- d3[shape_names]
dist3 <- dist(x3)
```

```

tree3 <- hclust(dist3, method = "ward.D2")

# Average the pocket silhouette scores for each candidate k.
ks <- 2:8
silhouette_mean <- numeric(length(ks))
for (i in 1:length(ks)) {
  groups <- cutree(tree3, k = ks[i])
  pocket_silhouette <- cluster::silhouette(groups, dist3)
  silhouette_mean[i] <- mean(pocket_silhouette[, "sil_width"])
}
data.frame(k = ks, Average_silhouette = silhouette_mean)

```

```

##   k Average_silhouette
## 1 2          0.3113
## 2 3          0.2879
## 3 4          0.2882
## 4 5          0.2872
## 5 6          0.2995
## 6 7          0.3086
## 7 8          0.2958

```

```
best_k <- ks[which.max(silhouette_mean)]
```

Columns: k is the number of groups; Average_silhouette is the average score from -1 to +1, not a percentage.

```

# Contrast small score differences with their absolute strength.
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(ks, silhouette_mean, type = "b", pch = 16,
     xlab = "k", ylab = "Average silhouette", main = "Default scale")
plot(ks, silhouette_mean, type = "b", pch = 16, ylim = c(-1, 1),
     xlab = "k", ylab = "Average silhouette", main = "Full scale [-1, 1]")
abline(h = 0.25, col = "#D95F02", lty = 2)
abline(h = 0.5, col = "#33A02C", lty = 2)

```

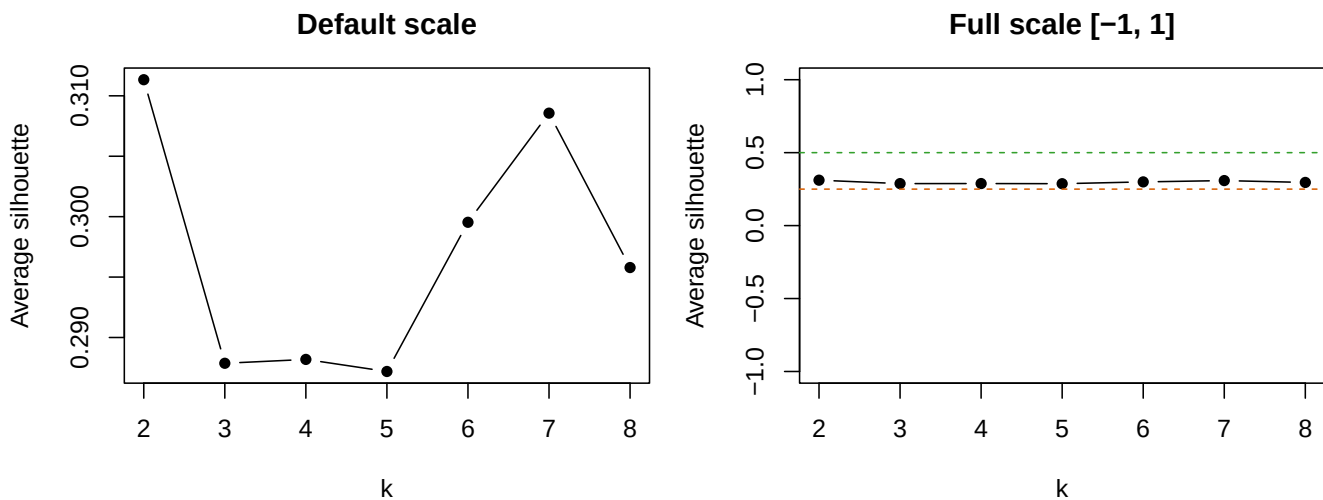


Figure 3: The same scores on the default and full theoretical scales.

```
par(mfrow = c(1, 1))
```

Optimal k? High score? $k = 2$ is best at 0.311, but separation is weak. The default scale highlights small differences; the full scale shows all scores are modest, well below 0.5. The reference lines are guides, not significance tests.

4.2 Bootstrap stability: $k = 4$, then 2 and 3

Bootstrap asks whether groups survive a changed sample. We draw 600 indices with replacement, rebuild Ward, and match each original group to the new group with maximum **Jaccard overlap**: shared members divided by all distinct members. It ranges from 0 (no overlap) to 1 (identical sets).

We compare only sampled identities, counting duplicates once in the overlap sets. This avoids penalizing absent pockets. All copies still enter clustering. Each group gets its best match; matching is not forced to be one-to-one.

```
# Run 500 reproducible resamples at each k.
B <- 500
set.seed(22518830)
stability <- list()
stability_table <- data.frame()
for (k in c(4, 2, 3)) {
  original <- cutree(tree3, k = k)
  scores <- matrix(NA_real_, B, k)
  for (b in 1:B) {
    # Resample and rebuild groups in the same shape space.
    ids <- sample(1:nrow(x3), nrow(x3), replace = TRUE)
    bootstrap_tree <- hclust(dist(x3[ids, ]), method = "ward.D2")
    bootstrap_groups <- cutree(bootstrap_tree, k = k)
    present <- unique(ids)
    for (g in 1:k) {
      reference <- intersect(which(original == g), present)
      if (length(reference) == 0) next
      matches <- numeric(k)
      for (h in 1:k) {
        # Calculate overlap with each candidate group.
        candidate <- unique(ids[bootstrap_groups == h])
        shared <- length(intersect(reference, candidate))
        total <- length(union(reference, candidate))
        matches[h] <- shared / total
      }
      scores[b, g] <- max(matches)
    }
  }
}
# Average each group's best overlap across resamples.
mean_jaccard <- colMeans(scores, na.rm = TRUE)
stability[[as.character(k)]] <- mean_jaccard
stability_table <- rbind(stability_table,
  data.frame(k = k, Cluster = 1:k, Mean_Jaccard = mean_jaccard))
}
stability_table
```

```
##   k Cluster Mean_Jaccard
## 1 4       1      0.5104
## 2 4       2      0.3917
## 3 4       3      0.5633
## 4 4       4      0.4572
```

```
## 5 2      1      0.7135
## 6 2      2      0.5970
## 7 3      1      0.4902
## 8 3      2      0.4760
## 9 3      3      0.5979
```

```
aggregate(Mean_Jaccard ~ k, data = stability_table, FUN = mean)
```

```
##   k Mean_Jaccard
## 1 2      0.6553
## 2 3      0.5214
## 3 4      0.4806
```

Columns: `k` identifies the partition; `Cluster` is its original group; `Mean_Jaccard` is its mean best overlap over 500 resamples. The final table averages across groups, weighting each group equally. Missing comparisons are omitted if a group has no sampled member.

Answer: `k = 4` scores range from 0.392 to 0.563, showing instability. `k = 2` gives 0.714 and 0.597, better but limited. `k = 3` gives 0.49, 0.476, 0.598. None reaches the usual 0.75 mean-Jaccard stability guideline. We retain four groups in session 4 only for the requested comparison.

4.3 Session 3 summary

Same question? No. Silhouette measures observed separation; bootstrap measures reproducibility under resampling. Both favor `k = 2` here, but neither strongly supports clearly separated natural groups.

5 Session 4 — Clustering, PCA, and heatmap

5.1 Repeat shape clustering and add scaled shape-plus-size clustering

We compare the same pockets at `k = 4`. The first tree exactly repeats the previous unscaled ratios; the second adds size and therefore requires scaling.

```
# Reload, recompute Rg, and prepare the two measurement spaces.
d4 <- read.csv("data_NS.csv", stringsAsFactors = TRUE)
d4$Rg <- sqrt(d4$Eigval.1 + d4$Eigval.2 + d4$Eigval.3)
x4 <- d4[shape_names]
z4 <- scale(d4[five_names])
dist_shape <- dist(x4)
dist_five <- dist(z4)

# Build both Ward partitions and check the repeated one is unchanged.
tree_shape <- hclust(dist_shape, method = "ward.D2")
tree_five <- hclust(dist_five, method = "ward.D2")
cluster_shape <- cutree(tree_shape, k = 4)
cluster_five <- cutree(tree_five, k = 4)
identical(cluster_shape, cutree(tree3, k = 4))
```

```
## [1] TRUE
```

```
cor(d4$Num_Atoms, d4$Rg)
```

```
## [1] 0.8297
```

Size weighting: Atom count and `Rg` correlate at about 0.83, partly repeating size information. Including both gives size more weight than using one descriptor. Scaling equalizes variances, not independent biological features; the shape ratios also contain redundancy.

5.2 Rebuild PCA1 and PCA2; show Origin and both clusterings

PCA summarizes the measurement spaces; cluster colors describe their groups. Origin remains supplementary in both PCAs.

```
# Repeat the PCAs from session 1 without using group labels to define axes.
pca1_4 <- PCA(d4[c(shape_names, "Origin")], scale.unit = FALSE,
              quali.sup = 4, graph = FALSE)
pca2_4 <- PCA(d4[c(five_names, "Origin")], scale.unit = TRUE,
              quali.sup = 6, graph = FALSE)
```

```
# Show known Origins on the shape-plus-size scores.
plot(pca2_4, choix = "ind", habillage = 6, label = "none",
     title = "Shape + size PCA: Origin")
```

Shape + size PCA: Origin

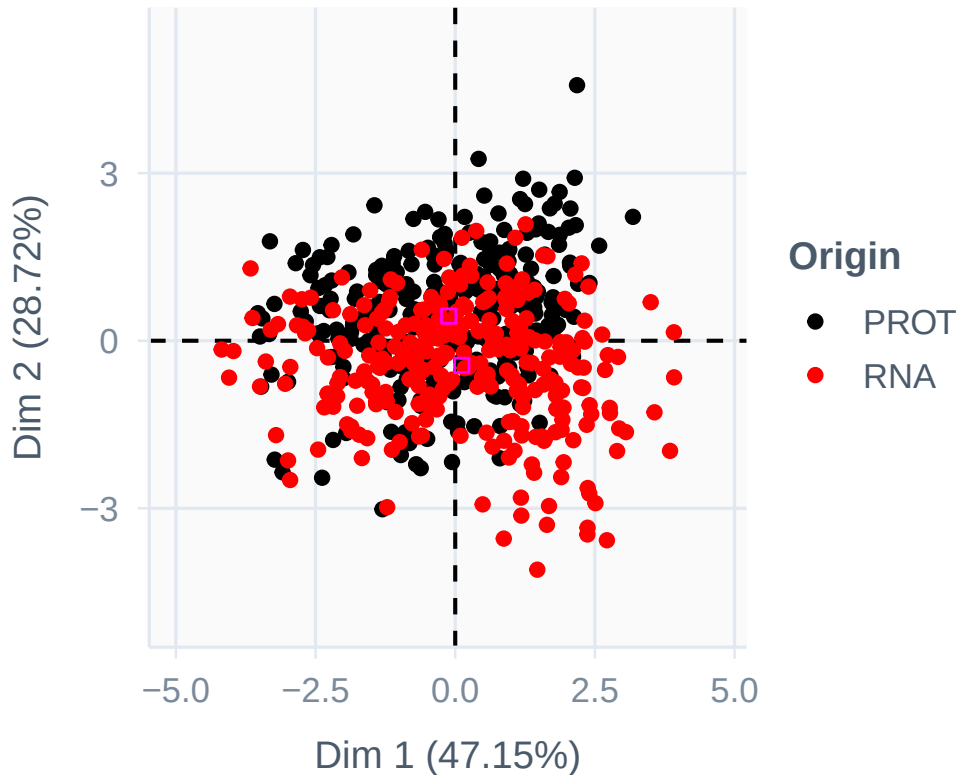


Figure 4: Supplementary Origin on the scaled shape-plus-size PCA.

Origin: RNA and proteins still overlap, but show a shift along size-related variation. RNA pockets have more atoms and larger mean Rg.

We now keep the same pure-shape coordinates and change only the colors. This tests whether the size-influenced groups are visible using shape alone.

```
# Use the same shape-only scores in both panels.
pc1 <- pca1_4$ind$coord[, 1]
pc2 <- pca1_4$ind$coord[, 2]
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(pc1, pc2, col = cluster_colors[cluster_five], pch = 16, cex = 0.5,
```

```

xlab = "PC1 (59.74%)", ylab = "PC2 (39.52%)",
main = "Five-variable groups")
legend("topright", legend = 1:4, col = cluster_colors, pch = 16, bty = "n")
plot(pc1, pc2, col = cluster_colors[cluster_shape], pch = 16, cex = 0.5,
xlab = "PC1 (59.74%)", ylab = "PC2 (39.52%)",
main = "Shape-only groups")
legend("topright", legend = 1:4, col = cluster_colors, pch = 16, bty = "n")

```

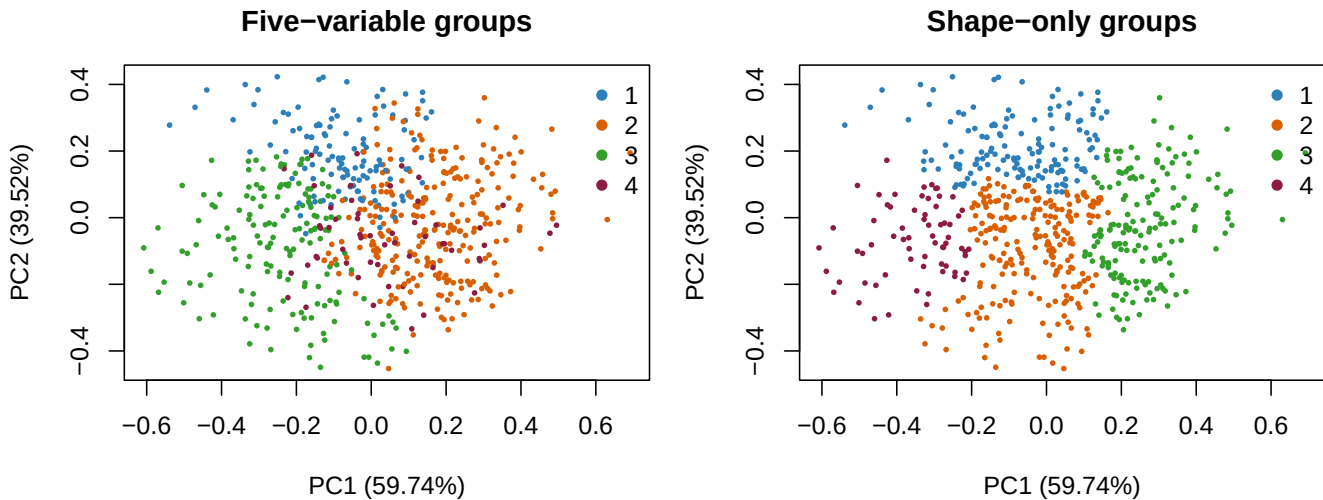


Figure 5: Identical shape coordinates colored by the five-variable and shape-only groups.

```

par(mfrow = c(1, 1))

```

Visible size trace? Partly: five-variable groups show some shape differences but overlap more. Their very large-pocket group does not form a distinct shape region. Shape-only colors form clearer regions because those groups were built from shape. Shape cannot fully recover the organization added by size.

5.3 Heatmap and numerical comparison

Rows are pockets; columns are the five measurements. Values are standardized: 0 is the mean, +1 is one SD above, and -1 one SD below. Blue means lower, white near average, and red higher. The five-variable Ward tree orders rows; side bars identify Origin and both group assignments.

```

# Match the measurements and category annotations by pocket row.
annotation <- data.frame(Origin = d4$Origin,
                          Cluster_five = factor(cluster_five),
                          Cluster_shape = factor(cluster_shape))
rownames(z4) <- paste0("Pocket_", 1:nrow(d4))
rownames(annotation) <- rownames(z4)
annotation_colors <- list(
  Origin = origin_colors,
  Cluster_five = setNames(cluster_colors, 1:4),
  Cluster_shape = setNames(cluster_colors, 1:4))

# Reuse the five-variable tree and retain the stated column order.
pheatmap(z4, cluster_rows = tree_five, cluster_cols = FALSE,
          annotation_row = annotation, annotation_colors = annotation_colors,
          show_rownames = FALSE, border_color = NA,
          color = hcl.colors(100, "Blue-Red 3"), fontsize = 9)

```

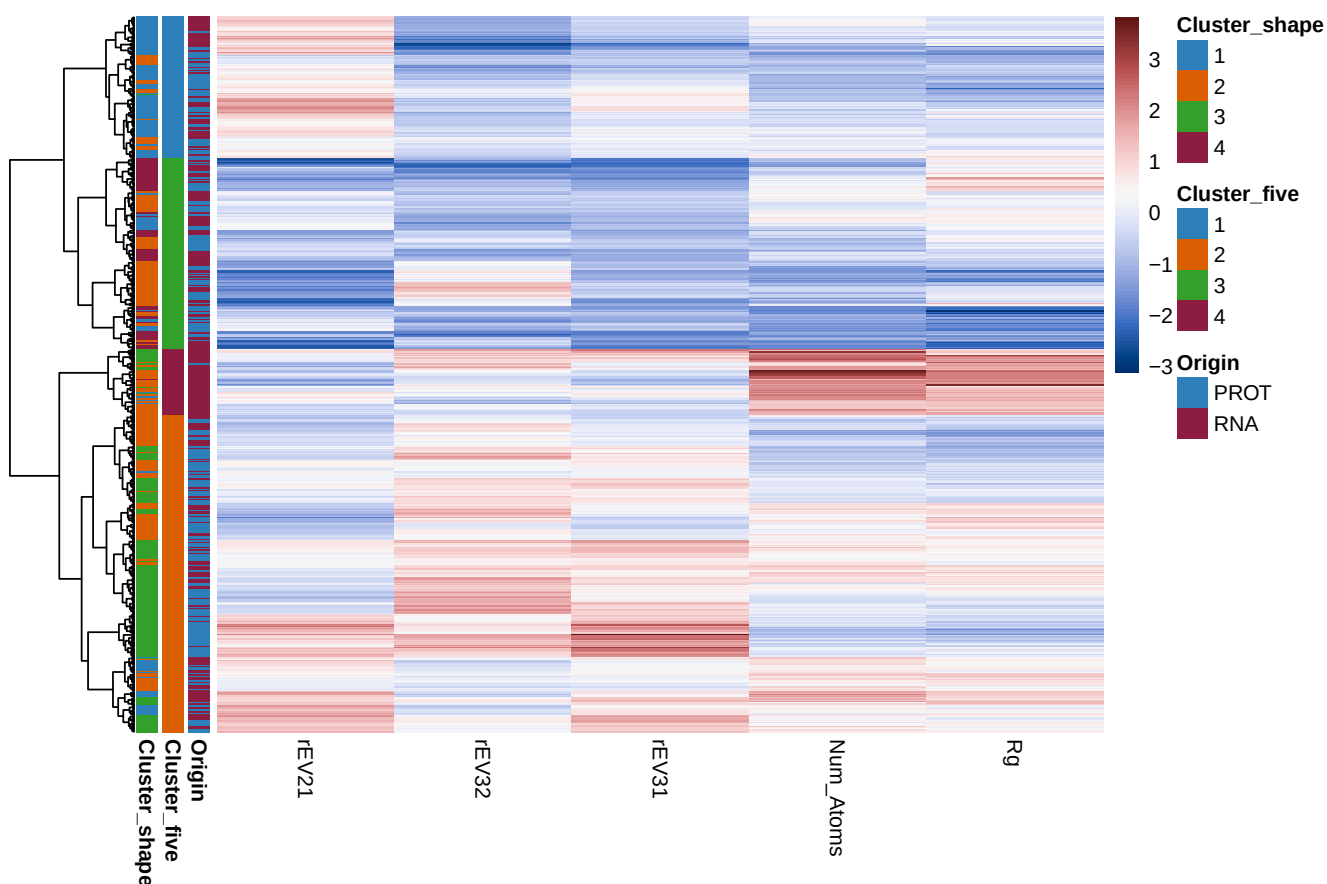


Figure 6: Standardized measurements ordered by the five-variable Ward tree.

Do blocks line up? Five-variable groups form contiguous blocks because their own tree orders the rows. Shape-only groups are more mixed; Origin is also mixed. Matching colors across the two group bars do not automatically mean matching groups.

```
# Count shared members: shape groups in rows, five-variable groups in columns.
cross_table <- table(Shape = cluster_shape, Five = cluster_five)
cross_table
```

```
##      Five
## Shape  1  2  3  4
##      1 92 27 21  5
##      2 26 88 73 32
##      3  1 151  0 17
##      4  0  0 66  1
```

```
# Convert shared counts into Jaccard overlaps for every group pair.
jaccard_matrix <- matrix(0, 4, 4)
rownames(jaccard_matrix) <- paste("Shape", 1:4)
colnames(jaccard_matrix) <- paste("Five", 1:4)
for (i in 1:4) {
  for (j in 1:4) {
    shared <- cross_table[i, j]
    total <- sum(cross_table[i, ]) + sum(cross_table[, j]) - shared
    jaccard_matrix[i, j] <- shared / total
  }
}
```

```

}
jaccard_matrix

##           Five 1   Five 2   Five 3   Five 4
## Shape 1 0.534884 0.07031 0.07394 0.025641
## Shape 2 0.083333 0.22166 0.23856 0.132231
## Shape 3 0.003484 0.53169 0.00000 0.082126
## Shape 4 0.000000 0.00000 0.40994 0.008264

# Find the best match for each shape group.
best_matches <- data.frame(Shape_cluster = 1:4,
                           Best_five_cluster = integer(4),
                           Best_Jaccard = numeric(4))
for (i in 1:4) {
  best_matches$Best_five_cluster[i] <- which.max(jaccard_matrix[i, ])
  best_matches$Best_Jaccard[i] <- max(jaccard_matrix[i, ])
}
best_matches

##   Shape_cluster Best_five_cluster Best_Jaccard
## 1             1                 1      0.5349
## 2             2                 3      0.2386
## 3             3                 2      0.5317
## 4             4                 3      0.4099

# Compare the complete partitions using ARI.
comparison <- fpc::cluster.stats(dist_shape, cluster_shape, cluster_five,
                                compareonly = TRUE)
agreement <- comparison$corrected.rand
agreement

## [1] 0.2576

```

Tables: The contingency entries count shared pockets. Jaccard entries give shared membership divided by combined distinct membership, between 0 and 1. `Shape_cluster` identifies the old group, `Best_five_cluster` its highest-overlap new group, and `Best_Jaccard` that overlap. Numbers are arbitrary, so the contingency diagonal has no special meaning.

Numerical answer: Best overlaps are 0.535, 0.239, 0.532, 0.41; ARI is 0.258. Agreement is partial: adding size changes membership, not just group names.

5.4 Final summary connecting the sessions

Session 1 separated shape from size and demonstrated why scaling matters. Session 2 showed that linkage changes memberships, with Ward avoiding strong chaining. Session 3 found weak separation and limited reproducibility of four shape groups. Session 4 showed that standardized size adds a different grouping signal. Together, the results support overlapping shape variation plus size variation, rather than four firmly established natural shape groups or two completely separate Origins.

6 Going further — Personal reflection

6.1 Archetype rule and comparison with both clusterings

Limitation: The complete CSV and detailed QMD named in `document.pdf` are absent. If the CSV is unavailable, we reconstruct the article's rule; these are not independently supplied labels. Archetypes are median-threshold

categories, not Ward results: high/high ratios give sphere-like, high/low disk-like, low/high rod-like, and low/low strongly anisotropic shapes.

```
# Estimate pooled thresholds and assign all four ratio combinations.
median21 <- median(d4$rEV21)
median32 <- median(d4$rEV32)
data.frame(Variable = c("rEV21", "rEV32"),
           Median_threshold = c(median21, median32))

##   Variable Median_threshold
## 1    rEV21           0.5923
## 2    rEV32           0.5751

high21 <- d4$rEV21 >= median21
high32 <- d4$rEV32 >= median32
rule_archetype <- rep("Strongly anisotropic", nrow(d4))
rule_archetype[high21 & high32] <- "Sphere-like"
rule_archetype[high21 & !high32] <- "Disk-like"
rule_archetype[!high21 & high32] <- "Rod-like"

# Load supplied labels if available, matching pocket identities first.
archetype <- rule_archetype
label_source <- "Reconstructed median-rule labels"
complete_path <- "../data/2026-09-09_data_PROT_RNA_TP_seance4_complet.csv"
if (file.exists(complete_path)) {
  complete <- read.csv(complete_path)
  key <- paste(d4$PDB_ID, d4$Pocket)
  complete_key <- paste(complete$PDB_ID, complete$Pocket)
  index <- match(key, complete_key)
  stopifnot("Archetype" %in% names(complete), !anyNA(index),
            !anyDuplicated(complete_key), !anyNA(complete$Archetype[index]))
  archetype <- complete$Archetype[index]
  label_source <- "Archetype labels loaded from the complete file"
}
label_source
```

```
## [1] "Reconstructed median-rule labels"
```

Variable names each ratio; Median_threshold gives its dividing value. High means at least the median. Identity matching avoids comparing unrelated rows if the complete file has a different order.

```
# Compare each clustering with the rule; integer labels preserve membership.
archetype_labels <- as.integer(factor(archetype))
shape_comparison <- fpc::cluster.stats(dist_shape, cluster_shape,
                                       archetype_labels, compareonly = TRUE)
five_comparison <- fpc::cluster.stats(dist_shape, cluster_five,
                                       archetype_labels, compareonly = TRUE)
shape_archetype_ari <- shape_comparison$corrected.rand
five_archetype_ari <- five_comparison$corrected.rand
data.frame(Clustering = c("Three ratios", "Five variables"),
           ARI_with_Archetype = c(shape_archetype_ari, five_archetype_ari))

##           Clustering ARI_with_Archetype
## 1   Three ratios           0.4002
## 2 Five variables           0.2898
```

Clustering identifies the analysis; ARI_with_Archetype measures its agreement with the categories. Shape-only ARI is 0.4 versus 0.29 with size. Neither reproduces the rule exactly; shape-only agrees more because the rule excludes size.

6.2 Frequencies and shape-versus-size effect sizes

```
# Compare category counts and percentages within each Origin.
frequency <- table(Archetype = archetype, Origin = d4$Origin)
frequency
```

```
##              Origin
## Archetype    PROT RNA
## Disk-like      74  91
## Rod-like       80  85
## Sphere-like    86  49
## Strongly anisotropic 60 75
```

```
100 * prop.table(frequency, margin = 2)
```

```
##              Origin
## Archetype    PROT  RNA
## Disk-like     24.67 30.33
## Rod-like      26.67 28.33
## Sphere-like   28.67 16.33
## Strongly anisotropic 20.00 25.00
```

```
# Express each RNA-minus-protein mean difference in pooled SD units.
effect_table <- data.frame(Variable = five_names, RNA_mean = numeric(5),
                           PROT_mean = numeric(5), Cohen_d = numeric(5))
for (i in 1:5) {
  rna_values <- d4[d4$Origin == "RNA", five_names[i]]
  protein_values <- d4[d4$Origin == "PROT", five_names[i]]
  pooled_sd <- sqrt((var(rna_values) + var(protein_values)) / 2)
  effect_table$RNA_mean[i] <- mean(rna_values)
  effect_table$PROT_mean[i] <- mean(protein_values)
  effect_table$Cohen_d[i] <-
    (mean(rna_values) - mean(protein_values)) / pooled_sd
}
effect_table
```

```
##   Variable RNA_mean PROT_mean Cohen_d
## 1   rEV21   0.5723   0.6000 -0.1645
## 2   rEV32   0.5475   0.6002 -0.2886
## 3   rEV31   0.3090   0.3596 -0.3719
## 4 Num_Atoms 244.7133 176.3533 0.8942
## 5      Rg    9.2485   8.7590 0.4116
```

Tables: Frequency rows are categories; columns are Origins. Percentages divide each column by 300. **Variable** names a measurement; **RNA_mean** and **PROT_mean** are original-unit averages; **Cohen_d** is their standardized difference. Positive d means higher in RNA. Equal group sizes allow the simple pooled-SD formula above.

Article comparison: Counts reproduce Table 3: sphere-like 49 RNA/86 protein, disk-like 91/74, strongly anisotropic 75/60, rod-like 85/80. Globally, disk-like and rod-like each account for 27.5%, the others 22.5%. Proteins have more sphere-like pockets; RNA has more disk-like and strongly anisotropic pockets; rod-like frequencies are similar.

The d values reproduce Table 1: shape -0.165/-0.289/-0.372, atom count +0.894, and Rg +0.412. Shape shifts are smaller than the atom-count difference. RNA pockets are larger on average, while the shape landscape remains shared.

6.3 Bootstrap stability of the median rule

Following the article, we resample 300 protein and 300 RNA pockets separately 1,000 times. Each time we recompute thresholds, classify **all original pockets**, and record ARI and the fraction changing category.

```
# Prepare reproducible Origin-stratified samples and result vectors.
protein_ids <- which(d4$Origin == "PROT")
rna_ids <- which(d4$Origin == "RNA")
original_rule_labels <- as.integer(factor(rule_archetype))
B_rule <- 1000
set.seed(20260910)
rule_ari <- numeric(B_rule)
reclassified <- numeric(B_rule)
for (b in 1:B_rule) {
  # Estimate pooled medians from the resampled pockets.
  sampled_protein <- sample(protein_ids, length(protein_ids), replace = TRUE)
  sampled_rna <- sample(rna_ids, length(rna_ids), replace = TRUE)
  ids <- c(sampled_protein, sampled_rna)
  new_median21 <- median(d4$rEV21[ids])
  new_median32 <- median(d4$rEV32[ids])

  # Apply new boundaries to all 600 original pockets.
  high21 <- d4$rEV21 >= new_median21
  high32 <- d4$rEV32 >= new_median32
  new_labels <- rep("Strongly anisotropic", nrow(d4))
  new_labels[high21 & high32] <- "Sphere-like"
  new_labels[high21 & !high32] <- "Disk-like"
  new_labels[!high21 & high32] <- "Rod-like"
  comparison <- fpc::cluster.stats(dist_shape, original_rule_labels,
    as.integer(factor(new_labels)), compareonly = TRUE)
  rule_ari[b] <- comparison$corrected.rand
  reclassified[b] <- mean(new_labels != rule_archetype)
}

# Summarize agreement and percentage changing category.
bootstrap_summary <- rbind(
  ARI = quantile(rule_ari, c(0.025, 0.5, 0.975)),
  Reclassified_percent = 100 * quantile(reclassified, c(0.025, 0.5, 0.975)))
bootstrap_summary
```

```
##                2.5%    50%   97.5%
## ARI              0.8085 0.9261 0.9868
## Reclassified_percent 0.5000 2.8333 7.6667
```

Columns: 2.5%-97.5% give the central 95% range; 50% is the median. Rows measure ARI agreement and percentage reclassified.

Answer: Median ARI is 0.926 [0.809, 0.987]; 2.8% change category, close to the article's 0.92 [0.82, 0.98] and 3%. Stable medians give reproducible categories; Ward groups were less stable. Their sampled-group Jaccard and the rule's all-pocket ARI are different statistics. Stable thresholds do not prove naturally separated groups.

Source: Ziani et al. (2026), supplied `article.pdf`, Tables 1 and 3 and bootstrap methods.